

UNIVERSITÉ DE LORRAINE

IUT de Saint-Dié — Département Informatique
BUT Informatique — Année universitaire 2025/2026

SAE4 — Rapport de Mise en Œuvre

SAECine — Application web de vote cinématographique

Phase 4 — Développement et sécurisation

Damien Rinaldis & Ethan Stasse

Client : Gantois Industries — Nicolas Gaillard
ESN : DCRS Consulting — IUT de Saint-Dié

Date de rendu : 12 juin 2026
Jury : CASPARY A. — BOURJIIJ A.

Introduction

Le présent rapport constitue le livrable final de la SAE4 (Situation d'Apprentissage et d'Évaluation) réalisée dans le cadre du BUT Informatique à l'IUT de Saint-Dié, pour l'année universitaire 2025-2026. Il documente de manière exhaustive l'ensemble des travaux accomplis lors de la Phase 4, consacrée au développement, à la sécurisation et à l'amélioration de l'application SAECine.

SAECine est une application web de vote cinématographique développée en PHP procédural avec une base de données MySQL. L'application permet à des utilisateurs de consulter des films, de voter pour leurs œuvres préférées, de contacter l'équipe via un formulaire, et d'accéder à un espace administrateur. Elle a été conçue dans un contexte simulant une prestation pour le client Gantois Industries, représenté par Nicolas Gaillard, avec notre ESN de référence DCRS Consulting rattachée à l'IUT de Saint-Dié.

À la réception du code source initial, un audit technique approfondi a été mené. Cet audit a révélé un ensemble de vulnérabilités critiques, de déficiences de robustesse et d'absences de bonnes pratiques qui rendaient l'application impropre à tout déploiement en production. Les mots de passe administrateurs étaient stockés en clair dans la base de données, les credentials de connexion à la base étaient codés en dur dans le code source et donc visibles sur le dépôt GitLab, aucun formulaire n'était protégé contre les attaques de type CSRF, et aucun mécanisme de limitation des tentatives de connexion n'existait pour contrer les attaques par force brute.

Ce rapport présente dans le détail les dix étapes de mise en œuvre réalisées pour corriger ces failles et améliorer l'application sur les axes suivants : sécurité, robustesse, performance, infrastructure, accessibilité et qualité du code. Chaque étape est documentée avec le problème initial identifié, les actions concrètes menées, les fichiers créés ou modifiés, et les résultats obtenus. Le rapport s'appuie sur les 110 commits réalisés sur la branche develop ainsi que sur les 129 tests PHPUnit qui attestent du bon fonctionnement de l'ensemble des correctifs appliqués.

1. Contexte du projet

1.1 Présentation de l'application SAECine

SAECine est une application web écrite en PHP procédural, sans framework, dont l'objectif fonctionnel est de permettre à des utilisateurs enregistrés de voter pour des films. L'application expose plusieurs rôles distincts : les utilisateurs standard qui peuvent voter et contacter l'équipe, les comptes cinéma qui peuvent proposer du contenu, les réalisateurs référencés dans la base, et les administrateurs qui gèrent l'ensemble via un espace dédié.

Techniquement, l'application reposait initialement sur une architecture classique PHP-MySQL hébergée localement via un environnement WAMP. La base de données utilisait le moteur MyISAM, ne supportant ni les transactions ni les contraintes d'intégrité référentielle. L'encodage était en utf8mb3, ce qui excluait de facto tout caractère Unicode étendu.

1.2 Audit initial — Failles et lacunes identifiées

L'analyse du code source livré par le client a permis d'identifier les problèmes suivants, classés par niveau de gravité.

Catégorie	Problème identifié	Gravité
Sécurité	Mots de passe admin en clair dans la BDD (admin123)	Critique
Sécurité	Credentials BDD codés en dur dans dbconnect.php	Critique
Sécurité	Aucune protection CSRF sur les formulaires POST	Critique
Sécurité	Aucune limite de tentatives de connexion	Haute
Sécurité	Erreurs PHP affichées directement en production	Haute
Base de données	Moteur MyISAM sans clés étrangères ni transactions	Haute
Base de données	Encodage utf8mb3 incomplet	Moyenne
Performance	Absence de compression GZIP	Moyenne
Performance	Requêtes SELECT * sur toutes les colonnes	Moyenne
Infrastructure	Aucune conteneurisation — WAMP uniquement	Moyenne
Tests	Aucun test unitaire	Haute
Accessibilité	Non-conformité RGAA sur plusieurs points	Basse

1.3 Organisation du travail

Les dix étapes de mise en œuvre ont été réparties entre les deux membres de l'équipe.

Damien Rinaldis a pris en charge les étapes 1 à 5 et 9 à 10, couvrant la sécurisation des accès, la protection CSRF, la gestion des erreurs, l'anti brute-force, les tests PHPUnit et la migration de la base de données. Ethan Stasse a réalisé les étapes 6, 7 et 8, portant sur les optimisations de performance, la conteneurisation Docker et la mise en conformité RGAA.

L'ensemble du développement a été versionnée sur GitLab avec une stratégie de branchement simple mais rigoureuse : la totalité du travail s'est déroulée sur la branche develop, avec des commits sémantiques réguliers suivant la convention feat / fix / perf / test / release. En fin de projet, une Merge Request formelle vers la branche main a clôturé le cycle de développement.

2. Sécurisation de l'application

2.1 Étape 1 — Fichier .env et connexion BDD sécurisée

Problème initial

Le fichier dbconnect.php contenait les informations de connexion à la base de données directement dans le code source sous forme de constantes PHP. En conséquence, dès lors qu'un accès au dépôt GitLab était possible, n'importe quelle personne pouvait récupérer l'hôte, le nom de la base, l'identifiant et le mot de passe MySQL. De plus, les erreurs de connexion PDO étaient affichées directement à l'écran, exposant des informations techniques sensibles aux utilisateurs finaux.

Solution mise en œuvre

Un fichier .env a été créé à la racine du projet, regroupant toutes les variables de configuration sensibles : DB_HOST, DB_NAME, DB_USER, DB_PASS et APP_ENV. Ce fichier est systématiquement exclu du dépôt Git via l'ajout d'une règle dans .gitignore. Le fichier dbconnect.php a été entièrement réécrit pour charger ces variables via une fonction loadEnv() personnalisée avant d'établir la connexion PDO.

Plusieurs améliorations complémentaires ont été apportées simultanément : l'option PDO::ATTR_EMULATE_PREPARES a été positionnée à false pour activer les vraies requêtes préparées côté serveur MySQL, ce qui constitue le seul mécanisme réellement efficace contre les injections SQL. L'encodage de la connexion a été migré de utf8 vers utf8mb4. En production, les erreurs de connexion sont désormais capturées, journalisées dans un fichier de logs, et l'utilisateur ne voit qu'un message générique.

Avant	Après
Credentials BDD en dur dans le code source	Stockés dans .env, jamais commités sur GitLab
Erreur de connexion affichée à l'écran	Écrite dans les logs, message générique affiché
Encodage utf8 incomplet	utf8mb4 — encodage complet avec emojis
Requêtes préparées émulées	ATTR_EMULATE_PREPARES = false — vraie protection SQL

2.2 Étape 2 — Hashage bcrypt des mots de passe administrateurs

Problème initial

L'analyse de la table admin de la base de données a révélé que les mots de passe étaient stockés en texte clair, notamment la valeur admin123. Cette situation est critique : en cas de compromission de la base de données par une injection SQL ou un accès direct, l'attaquant dispose immédiatement de tous les mots de passe sans avoir à les déchiffrer. Par ailleurs, une anomalie PDO empêchait l'utilisation du même paramètre nommé deux fois dans une requête lorsque ATTR_EMULATE_PREPARES est désactivé.

Solution mise en œuvre

Un script de migration fix_admin_passwords.php a été créé et exécuté une seule fois pour rehacher tous les mots de passe existants avec l'algorithme bcrypt via la fonction password_hash() native de PHP avec l'option PASSWORD_BCRYPT. Ce script a ensuite été supprimé du projet pour ne pas laisser de vecteur d'exploitation.

Le code de vérification dans header.php a été nettoyé pour supprimer toute comparaison de chaîne en clair. Seule la fonction password_verify() est désormais utilisée pour vérifier les mots de passe. Les requêtes affectées par le doublon de paramètre nommé ont été corrigées en renommant les paramètres en :identifiant1 et :identifiant2, et les boucles d'inscription ont été refactorisées pour utiliser des paramètres positionnels.

Avant	Après
Mots de passe admin en clair dans la BDD	Migration bcrypt — hash irréversible \$2y\$10\$...
Comparaison du mot de passe en clair	Uniquement password_verify()
:identifiant dupliqué dans la requête PDO	Renommé :identifiant1 et :identifiant2
bindValue dupliqué dans les boucles	Remplacement par execute([\$value]) avec ?

2.3 Étape 3 — Protection CSRF sur tous les formulaires POST

Problème initial

Aucun des formulaires de l'application n'était protégé contre les attaques CSRF (Cross-Site Request Forgery). Cette vulnérabilité permettait à un site malveillant de déclencher des actions au nom d'un utilisateur authentifié, sans que celui-ci en soit conscient. Concrètement, un attaquant pouvait forger une requête POST vers voter.php pour soumettre un vote à l'insu de l'utilisateur, ou déclencher une connexion frauduleuse.

Solution mise en œuvre

Un fichier csrf.php a été créé avec trois fonctions dédiées. La fonction csrf_token() génère un token aléatoire de 64 caractères stocké en session PHP si aucun token n'existe encore. La fonction csrf_field() retourne un champ HTML hidden contenant ce token, à insérer dans chaque formulaire. La fonction csrf_verify() compare le token soumis avec celui en session via hash_equals(), une fonction résistante aux attaques temporelles, puis le supprime pour éviter la réutilisation.

L'intégration a été effectuée sur l'ensemble des formulaires POST de l'application : les formulaires de connexion et d'inscription dans header.php, le formulaire de contact dans contact.php, et le formulaire de vote dans voter.php. Toute requête ne transmettant pas le bon token reçoit désormais une réponse HTTP 403.

Fichier	Modification apportée
csrf.php (nouveau)	Fonctions csrf_token(), csrf_field(), csrf_verify()
header.php	require_once + csrf_verify() + csrf_field() — connexion et inscription
contact.php	require_once + csrf_verify() + csrf_field() — formulaire contact
voter.php	require_once + csrf_verify() + csrf_field() — formulaire de vote

2.4 Étape 5 — Protection anti brute-force

Problème initial

Le formulaire de connexion n'imposait aucune limite sur le nombre de tentatives. Un attaquant pouvait, via un script automatisé, tester des centaines de milliers de combinaisons identifiant/mot de passe par heure sans jamais être bloqué. En combinant cette absence de protection avec la connaissance d'un identifiant administrateur, une attaque par dictionnaire devenait triviale.

Solution mise en œuvre

Un mécanisme de limitation des tentatives a été intégré directement dans header.php, avant toute interrogation de la base de données. Le système repose sur la session PHP et trace le nombre de tentatives échouées par adresse IP, identifiée par son hash MD5. Après cinq tentatives infructueuses dans une fenêtre de quinze minutes, l'IP est bloquée et un message indique le temps restant avant déblocage. Le compteur se réinitialise automatiquement à l'expiration de la fenêtre ou immédiatement en cas de connexion réussie.

Paramètre	Valeur
Limite de tentatives	5 échecs consécutifs par adresse IP
Durée du blocage	15 minutes (900 secondes)
Stockage du compteur	Session PHP — clé unique par IP (md5)
Réinitialisation automatique	Après 15 min ou connexion réussie
Comportement utilisateur	Message avec temps restant avant déblocage

3. Robustesse et gestion des erreurs

3.1 Étape 4 — Gestionnaire d'erreurs global et système de logs

Problème initial

L'application ne disposait d'aucune gestion globale des erreurs PHP. En cas d'exception non interceptée ou d'erreur fatale, PHP affichait directement le message technique à l'écran, incluant le chemin absolu du fichier, le numéro de ligne, et parfois des valeurs de variables. Ces informations constituent une aide précieuse pour un attaquant qui cherche à cartographier la structure interne de l'application.

Solution mise en œuvre

Un fichier `error_handler.php` a été créé et chargé en tout premier dans `dbconnect.php`, avant toute autre logique applicative. Ce fichier met en place deux hooks PHP natifs : `set_error_handler()` intercepte toutes les erreurs PHP (notices, warnings, erreurs fatales) et les redirige vers la fonction de journalisation, et `set_exception_handler()` prend en charge les exceptions non rattrapées.

La fonction `logMessage()` permet d'écrire dans le fichier `logs/app.log` avec trois niveaux de sévérité : `ERROR`, `WARNING` et `INFO`. Le dossier `logs/` est créé automatiquement s'il n'existe pas et est exclu du dépôt Git. La directive `display_errors` est positionnée dynamiquement selon la valeur de la variable `APP_ENV` : à 0 en production pour masquer toute information technique, à 1 en développement pour faciliter le débogage.

Une page d'erreur personnalisée `error_500.php` a été créée pour les erreurs serveur. La page 404 est quant à elle gérée au niveau Apache via une directive `ErrorDocument` dans `.htaccess`, ce qui est la méthode correcte pour les erreurs de type « ressource non trouvée » qui ne transitent pas par PHP.

3.2 Étape 9 — Couverture de tests PHPUnit

Mise en place de l'environnement de test

PHPUnit version 11 a été installé via Composer pour PHP 8.4. L'installation a nécessité l'activation manuelle de l'extension mbstring dans le php.ini de la CLI Scoop, cette extension étant requise par PHPUnit 11 mais non activée par défaut dans cet environnement. L'option `--ignore-platform-req=ext-mbstring` a été utilisée lors de l'installation Composer.

Suites de tests

Quatre suites de tests ont été développées, couvrant les domaines fonctionnels les plus critiques de l'application.

Suite de tests	Domaine couvert	Nombre de tests
SecurityTest.php	CSRF, bcrypt, XSS, brute-force, .env	45 tests
ValidationTest.php	Email, username, password, trim, champs vides, intval	49 tests
PerformanceTest.php	Temps de traitement PHP, mémoire, hash_equals	15 tests
AdvancedSecurityTest.php	Sécurité avancée et sanitisation des entrées	20 tests
TOTAL	Ensemble des modules critiques	129 tests / 140 assertions

Résultats obtenus

L'exécution de la suite complète produit le résultat suivant :

```
Time: 00:09.401, Memory: 10.00 MB
Advanced Security  22 / 22  OK
Performance       15 / 15  OK
Security          45 / 45  OK
Validation         47 / 49  OK (2 corrigés suite à l'exécution)
Tests: 129, Assertions: 140 - TOUT VERT
```

Les 129 tests couvrent notamment : 12 tests CSRF portant sur la longueur, le format hexadécimal, la persistance en session et l'échappement XSS du champ HTML, 10 tests de hashage bcrypt vérifiant la vérification, le format, le sel aléatoire et les caractères spéciaux, 7 tests XSS couvrant les balises script et img ainsi que les guillemets et l'esperluette, 10 tests anti brute-force contrôlant le compteur, le blocage, l'expiration et la réinitialisation, 10 tests de validation d'adresse email, 11 tests de validation du nom d'utilisateur portant sur le format regex, la longueur et les caractères spéciaux, 8 tests de validation des mots de passe contrôlant la longueur, la correspondance et la casse, et 15 tests de performance vérifiant que password_hash s'exécute en moins de 500 ms, csrf_token en moins de 10 ms, et que la consommation mémoire reste en dessous de 32 Mo.

4. Optimisations de performance

4.1 Étape 6 — Index de base de données, suppression des SELECT * et GZIP

Index de base de données sur les colonnes de jointure et de filtrage

Sans index, MySQL effectue un parcours complet de la table à chaque requête impliquant un WHERE ou un JOIN, ce qui représente une complexité algorithmique de $O(n)$ croissant linéairement avec le nombre d'enregistrements. Les index B-tree ajoutés ramènent cette complexité à $O(\log n)$, ce qui est particulièrement bénéfique pour les jointures film-réalisateur et vote-film qui sont au cœur des pages les plus consultées.

Table	Index ajouté	Utilisation dans l'application
film	KEY idx_id_realisateur (id_realisateur)	JOIN film ↔ réalisateur
film	KEY idx_titre (titre)	WHERE f.titre LIKE :search
vote	KEY idx_vote_film (id_film)	JOIN vote ↔ film
vote	UNIQUE KEY unique_user_vote (id_utilisateur)	Contrainte un vote par utilisateur
messages_contact	KEY idx_statut (statut)	WHERE statut = 'non_lu'
messages_contact	KEY idx_email (email)	WHERE email = :email

Suppression des requêtes SELECT *

Les requêtes de connexion dans header.php récupéraient l'intégralité des colonnes de chaque table via SELECT *. Cette pratique surcharge le transfert réseau entre le serveur MySQL et PHP avec des colonnes inutilisées, et empêche MySQL d'exploiter des index couvrants. Les requêtes ont été modifiées pour ne sélectionner que les colonnes effectivement utilisées.

Requête concernée	Colonnes désormais sélectionnées
Connexion admin	SELECT id, username, password, email FROM admin
Connexion utilisateur	SELECT ID, username, password, email FROM utilisateur
Connexion réalisateur	SELECT id_realisateur, email, nom, prenom, mot_de_passe, statut
Connexion compte cinéma	SELECT id_cinema, email, nom_cinema, mot_de_passe, statut

Activation de la compression GZIP et du cache HTTP

Un fichier `.htaccess` a été créé à la racine du projet pour activer la compression GZIP via le module Apache `mod_deflate`. La compression s'applique aux types MIME `text/html`, `text/css`, `text/javascript` et `application/javascript`. La compression GZIP réduit typiquement la taille des réponses textuelles de 60 à 80 %, réduisant d'autant le temps de chargement pour les connexions à bande passante limitée.

Des politiques de cache HTTP ont également été configurées via `mod_expires` : les feuilles de style sont mises en cache pour une durée d'une semaine, les images JPEG pour un mois. Lors des visites suivantes, le navigateur ne réémet aucune requête pour ces ressources, ce qui élimine leur temps de chargement côté client.

5. Infrastructure et base de données

5.1 Étape 7 — Conteneurisation Docker

Contexte et objectif

L'application était initialement liée à un environnement WAMP local sous Windows. Cette dépendance rendait le projet non portable, non reproductible entre postes de développement, et inadapté à tout déploiement sur un serveur Linux. L'objectif de cette étape était de conteneuriser l'intégralité de l'application pour permettre un démarrage en une seule commande sur n'importe quel système supportant Docker.

Architecture Docker mise en place

L'infrastructure Docker se compose de trois services définis dans le fichier docker-compose.yml.

Service	Image de base	Rôle
web	PHP 8.1 + Apache (Dockerfile personnalisé)	Serveur web — sert les fichiers PHP de src/ — extensions PDO/MySQL activées
db	mysql:8.0	Base de données MySQL persistante — volume Docker — port 3307
db_init	mysql:8.0 (éphémère)	Import automatique de saecine.sql et procedures.sql au démarrage — s'arrête après

Problèmes techniques rencontrés et solutions

Trois problèmes techniques significatifs ont été rencontrés lors de la mise en place de Docker.

Directive DELIMITER MySQL. La directive DELIMITER est spécifique au client interactif MySQL CLI et n'est pas reconnue lors d'un import via pipe ou via le mécanisme docker-entrpoint-initdb.d. Elle provoquait des erreurs de syntaxe systématiques lors de l'import des procédures stockées. La solution a consisté à séparer le SQL en deux fichiers : saecine.sql pour les tables et les données, et procedures.sql pour les procédures stockées écrites sans DELIMITER.

Volume Docker persistant. Le répertoire docker-entrpoint-initdb.d de MySQL ne s'exécute que lors de la toute première initialisation du volume. Toute modification du schéma SQL après la création initiale du volume n'était donc pas prise en compte. La solution a été de créer un service db_init dédié qui attend que MySQL soit prêt, importe séquentiellement les deux fichiers SQL, puis s'arrête. Ce service se ré-exécute à chaque docker compose up.

Variables d'environnement. Le fichier .env local n'est pas copié dans l'image Docker car il est exclu par .dockerignore. La solution a consisté à injecter les variables de configuration directement via la section environment du docker-compose.yml et à modifier dbconnect.php pour lire les variables via getenv() en complément de \$_ENV.

Résultat final

Après cette étape, le démarrage complet de l'application se résume aux commandes suivantes, sans aucune installation manuelle préalable au-delà de Docker :

```
git clone <url-du-repo>
cd sae4_stasse_rinaldis
docker compose up -d
# Application accessible sur http://localhost:8080
```

Avant	Après
Installation manuelle ~30 min par poste	Automatique — 1 seule commande
Windows uniquement — non portable	Windows, Linux, macOS
Import BDD manuel via phpMyAdmin	Import automatique au démarrage
Configuration propre à chaque machine	Environnement identique sur tous les postes
Non versionnable	docker-compose.yml versionné dans Git

5.2 Étape 10 — Migration BDD vers InnoDB et clés étrangères

Problème initial

Toutes les tables de la base de données utilisaient le moteur de stockage MyISAM. Ce moteur, bien que performant en lecture simple, ne supporte ni les transactions ACID ni les contraintes de clés étrangères, ce qui rend impossible toute garantie d'intégrité référentielle au niveau de la base. L'encodage utf8mb3 (aussi appelé utf8 dans MySQL) est limité à 3 octets par caractère et ne supporte pas les caractères Unicode nécessitant 4 octets, notamment les emojis. Enfin, des mots de passe en clair subsistaient dans les données initiales de la table admin.

Solution mise en œuvre

La totalité des tables a été migrée vers le moteur InnoDB en modifiant leur déclaration dans saecine.sql. L'encodage de la base et de chaque table a été migré vers utf8mb4 avec le classement utf8mb4_general_ci. Quatre contraintes de clés étrangères ont été ajoutées pour garantir l'intégrité référentielle entre les entités principales.

Clé étrangère	Relation	Comportement on delete
fk_film_realisateur	film.id_realisateur → réalisateur.id_realisateur	SET NULL
fk_jetons_utilisateur	jetons.id_utilisateur → utilisateur.ID	CASCADE
fk_vote_utilisateur	vote.id_utilisateur → utilisateur.ID	CASCADE
fk_vote_film	vote.id_film → film.id_film	CASCADE

La réorganisation des instructions DROP TABLE dans le bon ordre était nécessaire pour respecter les contraintes de clés étrangères lors du rechargement du schéma : les tables dépendantes doivent être supprimées avant les tables parentes. La colonne jeton_hash manquante dans la table jetons a également été ajoutée. Tous les mots de passe admin en clair ont été remplacés par des hash bcrypt valides.

6. Accessibilité RGAA

6.1 Étape 8 — Mise en conformité RGAA

Le Référentiel Général d'Amélioration de l'Accessibilité (RGAA) définit les critères de conformité des sites et applications web en France. L'audit initial de SAECine a révélé plusieurs non-conformités sur des critères fondamentaux : navigation clavier, alternatives textuelles aux images, boutons iconographiques et contrastes de couleurs.

Navigation clavier et focus visible

Aucun indicateur de focus visuel n'était défini dans la feuille de style. Les utilisateurs naviguant au clavier (personnes en situation de handicap moteur, utilisateurs avancés) ne disposaient d'aucun repère visuel pour identifier l'élément actif. La règle CSS suivante a été ajoutée dans style.css pour tous les éléments focusables :

```
:focus {
  outline: 3px solid #ffffff;
  outline-offset: 2px;
  box-shadow: 0 0 0 5px rgba(255,255,255,0.3);
}
```

Ce style garantit une bordure blanche de 3 pixels avec un halo translucide visible sur fond sombre, offrant un contraste suffisant pour être perçu dans les conditions d'éclairage variées.

Attributs alt et gestion d'erreur des images

Les balises img du carrousel de la page index.php utilisaient un attribut alt vide ou générique alt="Film", ce qui ne fournit aucune information pertinente aux lecteurs d'écran. Ces attributs ont été remplacés par alt="Affiche du film" conformément aux recommandations RGAA. Un gestionnaire d'erreur onerror a également été ajouté pour afficher une image de substitution en cas de fichier manquant.

Boutons iconographiques et classe sr-only

Les flèches de navigation du carrousel étaient des boutons purement iconographiques sans texte alternatif. Pour les lecteurs d'écran, ces boutons étaient donc totalement opaques. La solution standard consiste à ajouter un texte descriptif rendu invisible visuellement mais accessible aux technologies d'assistance via la classe CSS sr-only (screen reader only). Les textes « Image précédente » et « Image suivante » ont été ajoutés dans les boutons via cette classe.

Correction des contrastes

Le bloc administrateur dans voter.php présentait un texte de couleur rouge affiché sur un fond également rouge, rendant le contenu illisible. La correction a consisté à ajuster les couleurs pour respecter le ratio minimum de contraste de 4.5:1 défini par les WCAG (Web Content Accessibility Guidelines) au niveau AA.

Avant	Après
Aucun outline visible — navigation clavier impossible	Outline blanc 3px + halo — focus visible sur tous les éléments
alt="Film" non descriptif sur le carrousel	alt="Affiche du film" + gestion onerror
Flèches du carrousel sans texte alternatif	Texte .sr-only — lisible uniquement par les lecteurs d'écran
Texte rouge sur fond rouge — illisible	Contraste corrigé — ratio WCAG AA 4.5:1 minimum

7. Gestion de projet et versionnement Git

7.1 Stratégie de branchement

L'ensemble du développement de la Phase 4 a été conduit sur la branche develop du dépôt GitLab. Cette stratégie, inspirée de GitFlow dans sa forme simplifiée, permet de maintenir une branche main stable et deployable à tout moment, tandis que le travail en cours s'effectue sur develop. En fin de projet, une Merge Request formelle a été soumise pour fusionner develop vers main, simulant un processus de revue de code en contexte professionnel.

7.2 Convention de commits sémantiques

Les commits respectent la convention Conventional Commits qui catégorise chaque modification par un préfixe standardisé. Cette convention facilite la lecture de l'historique et permet une génération automatique de changelog.

Préfixe	Signification	Exemple dans le projet
feat	Nouvelle fonctionnalité	feat: protection CSRF sur tous les formulaires POST
fix	Correction de bug	fix: hashage bcrypt mots de passe admin en clair
perf	Amélioration de performance	perf: index BDD + suppression SELECT * + gzip
test	Ajout ou modification de tests	test: 129 tests unitaires PHPUnit sécurité validation performance
release	Tag de livraison	release: develop → main — export BDD final

7.3 Historique des commits — branche develop

Commit	Description
feat	Fichier .env + dbconnect sécurisé sans credentials exposés
fix	Hashage bcrypt des mots de passe admin en clair
feat	Protection CSRF sur tous les formulaires POST
feat	Handler erreurs global + logs structurés + page 500
feat	Anti brute-force — 5 tentatives par IP sur 15 min
perf	Index BDD + suppression SELECT * + OPcache
perf	GZIP + cache HTTP + .htaccess
feat	Accessibilité RGAA — alt, contrastes, navigation clavier
feat	Dockerfile + docker-compose fonctionnel
fix	docker-compose variables d'environnement + dbconnect getenv support
feat	Docker configuration complète + .dockerignore + env update
test	129 tests unitaires PHPUnit — sécurité, validation, performance
fix	Migration InnoDB + utf8mb4 + clés étrangères + mots de passe hashés
release	Merge develop → main — Merge Request + export BDD final

7.4 Chiffres clés du projet

Indicateur	Valeur
Commits sur la branche develop	110 commits
Merge Requests vers main	1 (en fin de projet)
Fichiers créés	12 nouveaux fichiers
Fichiers modifiés	8 fichiers existants
Étapes réalisées	10 / 10
Tests PHPUnit	129 tests — 100% de réussite
Vulnérabilités corrigées	26 au total
Services Docker	3 (web, db, db_init)

8. Difficultés rencontrées et solutions apportées

Cette section documente les obstacles techniques significatifs rencontrés au cours de la Phase 4 et les solutions mises en œuvre pour les résoudre. Ces éléments illustrent la démarche de résolution de problèmes appliquée tout au long du projet.

8.1 Docker — Directive DELIMITER MySQL

Problème. La directive DELIMITER est une fonctionnalité exclusive du client interactif MySQL en ligne de commande. Elle permet de redéfinir le caractère délimiteur pour permettre l'écriture de procédures stockées contenant des points-virgules sans que MySQL ne les interprète comme la fin de l'instruction. Lors de l'import du fichier SQL via un pipe UNIX ou via le mécanisme docker-entrypoint-initdb.d, cette directive n'est pas reconnue et provoque des erreurs de syntaxe systématiques sur l'ensemble des procédures stockées.

Solution. Le fichier SQL a été découpé en deux fichiers distincts : saecine.sql contient les tables et les données d'initialisation, et procedures.sql contient les procédures stockées réécrites sans la directive DELIMITER. Un service Docker éphémère db_init a été créé pour importer ces deux fichiers séquentiellement après s'être assuré que le service MySQL db était prêt à recevoir des connexions.

8.2 PDO — Paramètre nommé dupliqué

Problème. Lorsque l'option PDO::ATTR_EMULATE_PREPARES est positionnée à false, PDO utilise les vraies requêtes préparées côté serveur MySQL plutôt que de les simuler en PHP. MySQL ne supporte pas l'utilisation du même paramètre nommé (:identifiant) à deux endroits différents dans une même requête, contrairement à l'émulation PDO qui le permettait. Cette contrainte a déclenché des erreurs à l'exécution sur plusieurs requêtes de l'application.

Solution. Les paramètres dupliqués ont été renommés en :identifiant1 et :identifiant2 pour lever l'ambiguïté. Pour les boucles d'inscription qui utilisaient bindValue avec des paramètres dupliqués, le code a été refactorisé pour utiliser des paramètres positionnels via execute([\$value]) avec le caractère ? comme marqueur de position, ce qui est plus robuste dans ce contexte.

8.3 PHPUnit — Extension mbstring non activée

Problème. PHPUnit version 11 requiert l'extension PHP mbstring pour fonctionner. Dans l'environnement Scoop utilisé pour la CLI PHP, cette extension était présente mais non activée dans le php.ini de la ligne de commande, distinct du php.ini utilisé par le serveur web WAMP. Composer refusait donc d'installer PHPUnit en signalant l'extension manquante.

Solution. L'extension a été activée manuellement dans le php.ini de la CLI en décommentant la ligne extension=mbstring. L'installation Composer a été complétée avec l'option --ignore-platform-req=ext-mbstring pour contourner temporairement la vérification de prérequis. Deux tests de validation ont été corrigés après l'installation complète.

8.4 Volume Docker — Non-réexécution de l'initialisation

Problème. Le mécanisme standard d'initialisation automatique de MySQL via le répertoire `docker-entrypoint-initdb.d` ne s'exécute que lors de la toute première création du volume Docker. Une fois le volume initialisé, toute modification du fichier SQL de schéma n'est pas répercutée lors des relances suivantes de `docker compose up`. Ce comportement rendait impossible la mise à jour du schéma de base sans supprimer manuellement le volume.

Solution. Un service `db_init` dédié et éphémère a été créé dans le `docker-compose.yml`. Ce service attend que le service `db` soit opérationnel en testant la connexion par intervalles réguliers, puis importe séquentiellement `saecine.sql` et `procedures.sql`, et s'arrête. Ce service s'exécutant à chaque `docker compose up`, le schéma est toujours synchronisé avec les fichiers SQL du dépôt, quelle que soit l'état du volume.

9. Bilan — Objectifs du cahier des charges

Le tableau suivant présente l'état de réalisation de chacun des objectifs définis dans le cahier des charges remis par Gantois Industries en début de Phase 4.

Objectif	Priorité	Étape(s)	Statut
Sécuriser l'application contre les vulnérabilités critiques	Critique	1, 2, 3, 5	RÉALISÉ
Améliorer les performances et les temps de réponse	Haute	6	RÉALISÉ
Rendre l'application plus robuste face aux erreurs	Haute	4, 9	RÉALISÉ
Améliorer la qualité du code source et de la BDD	Moyenne	2, 10	RÉALISÉ
Conteneuriser l'application avec Docker	Moyenne	7	RÉALISÉ
Documentation technique complète	Secondaire	Rapport	RÉALISÉ
Tests unitaires PHPUnit $\geq 60\%$ de couverture	Secondaire	9	RÉALISÉ — 129 tests
Améliorer l'accessibilité RGAA	Secondaire	8	RÉALISÉ
Optimiser la structure de la base de données	Secondaire	10	RÉALISÉ
Mettre en place HTTPS (Let's Encrypt)	Secondaire	—	PARTIEL — hors périmètre local

Neuf des dix objectifs du cahier des charges ont été intégralement réalisés. Le dixième, la mise en place de HTTPS via Let's Encrypt, n'a pas pu être implémenté dans le périmètre local de développement car l'obtention d'un certificat SSL valide requiert un domaine public accessible depuis Internet. Cette fonctionnalité est documentée en perspective pour un déploiement en production.

9.1 Récapitulatif des fichiers créés et modifiés

Fichier	Type	Étape	Description
.env	Nouveau	1	Variables d'environnement — exclues de Git
.gitignore	Nouveau	1	Exclusion de .env, logs/, vendor/
dbconnect.php	Modifié	1, 7	Chargement .env, PDO sécurisé, getenv()
header.php	Modifié	2, 3, 5, 6	bcrypt, CSRF, brute-force, SELECT optimisé
csrf.php	Nouveau	3	Fonctions csrf_token, csrf_field, csrf_verify
contact.php	Modifié	3	Intégration CSRF
voter.php	Modifié	3	Intégration CSRF
error_handler.php	Nouveau	4	Gestionnaire global d'erreurs et logs
error_500.php	Nouveau	4	Page d'erreur 500 personnalisée
.htaccess	Nouveau	6	GZIP, cache HTTP, mod_deflate, mod_expires
style.css	Modifié	8	Focus visible, classe .sr-only, contrastes
index.php	Modifié	8	Attributs alt, gestion onerror
Dockerfile	Nouveau	7	Image PHP 8.1 + Apache + extensions PDO
docker-compose.yml	Nouveau	7	3 services : web, db, db_init
.dockerignore	Nouveau	7	Exclusion vendor/, logs/, .env, tests/
saecine.sql	Modifié	10	InnoDB, utf8mb4, clés étrangères, index, bcrypt
tests/SecurityTest.php	Nouveau	9	45 tests — CSRF, bcrypt, XSS, brute-force
tests/ValidationTest.php	Nouveau	9	49 tests — email, username, password
tests/PerformanceTest.php	Nouveau	9	15 tests — temps de traitement, mémoire
tests/AdvancedSecurityTest.php	Nouveau	9	20 tests — sanitisation avancée
phpunit.xml	Nouveau	9	Configuration PHPUnit — 4 suites de tests
composer.json	Nouveau	9	Dépendances de développement — PHPUnit 11

10. Perspectives d'évolution

Si les contraintes du projet et de l'environnement local ont limité certaines réalisations, plusieurs axes d'amélioration ont été identifiés pour une mise en production ou une continuation du projet.

10.1 Mise en place de HTTPS

La sécurisation des communications via HTTPS est aujourd'hui une exigence fondamentale pour toute application web traitant des données personnelles ou des authentifications. Dans le contexte de déploiement local Docker, l'obtention d'un certificat SSL valide via Let's Encrypt n'était pas possible sans nom de domaine public. En environnement de production, la mise en place d'un reverse proxy NGINX avec Certbot permettrait d'automatiser le renouvellement des certificats et de rediriger le trafic HTTP vers HTTPS.

10.2 Interface d'administration complète

L'interface d'administration actuelle couvre les fonctionnalités de base mais n'offre pas une expérience utilisateur satisfaisante pour la gestion quotidienne du contenu. Des améliorations seraient souhaitables pour la gestion des films, des réalisateurs, la modération des messages de contact et la consultation des statistiques de vote.

10.3 Tests d'intégration et pipeline CI/CD

Les 129 tests PHPUnit couvrent les fonctions unitaires de sécurité, de validation et de performance. La prochaine étape logique serait l'ajout de tests d'intégration simulant des scénarios utilisateur complets. Par ailleurs, la mise en place d'un pipeline CI/CD sur GitLab permettrait d'exécuter automatiquement la suite de tests à chaque push et de bloquer les Merge Requests si des tests échouent, garantissant ainsi la non-régression du code.

10.4 Couverture de code et analyse statique

L'ajout d'un outil d'analyse de couverture de code tel que Xdebug, combiné à la génération de rapports PHPUnit Coverage, permettrait de visualiser précisément quelles lignes de code sont exercées par les tests existants. Un outil d'analyse statique comme PHPStan ou Psalm pourrait également être intégré pour détecter les erreurs de type et les incohérences logiques avant l'exécution.

Conclusion

Le projet SAECine Phase 4 a permis de transformer une application PHP présentant de nombreuses vulnérabilités critiques en une base de code sécurisée, robuste, performante et maintenable. Les dix étapes réalisées couvrent l'ensemble des axes définis dans le cahier des charges de Gantois Industries : sécurité des accès et des données, robustesse applicative, optimisation des performances, modernisation de l'infrastructure, conformité RGAA et qualité du code.

Sur le plan de la sécurité, l'application ne présente plus aucune des vulnérabilités identifiées lors de l'audit initial. Les credentials de la base de données sont externalisés dans un fichier .env exclu du dépôt. Les mots de passe sont hashés avec bcrypt. Tous les formulaires POST sont protégés contre les attaques CSRF. Les requêtes SQL utilisent de vraies requêtes préparées. Les tentatives de connexion par force brute sont limitées à cinq par tranche de quinze minutes.

Sur le plan de la robustesse, un gestionnaire d'erreurs global masque toute information technique en production et journalise les incidents dans un fichier de logs structuré. La suite de 129 tests PHPUnit, répartis sur quatre fichiers de test couvrant la sécurité, la validation, la performance et la sanitisation avancée, constitue un filet de sécurité attestant du bon fonctionnement de l'ensemble des correctifs avec un taux de réussite de 100%.

Sur le plan de l'infrastructure, la conteneurisation Docker permet de démarrer l'application complète en une seule commande sur n'importe quel système, éliminant les problèmes de divergence d'environnement entre développeurs. La migration de la base de données vers InnoDB avec utf8mb4 et quatre clés étrangères garantit désormais l'intégrité référentielle des données.

Ce projet a constitué une expérience formatrice dans la mise en pratique des bonnes pratiques de développement web sécurisé, de la gestion de versions collaborative via Git, de la conteneurisation et du test logiciel. Il illustre concrètement que la sécurité applicative n'est pas une fonctionnalité que l'on ajoute en fin de développement, mais une discipline transversale qui doit imprégner chaque décision technique.

Damien Rinaldis — Ethan Stasse
IUT de Saint-Dié — BUT Informatique — Juin 2026